The Anatomy of an AI Agent

for Developers

Rob Windsor
rob@robwindsor.com

About Me



Rob Windsor

.NET/Microsoft 365 developer, trainer, author
Microsoft MVP | Microsoft 365 Development

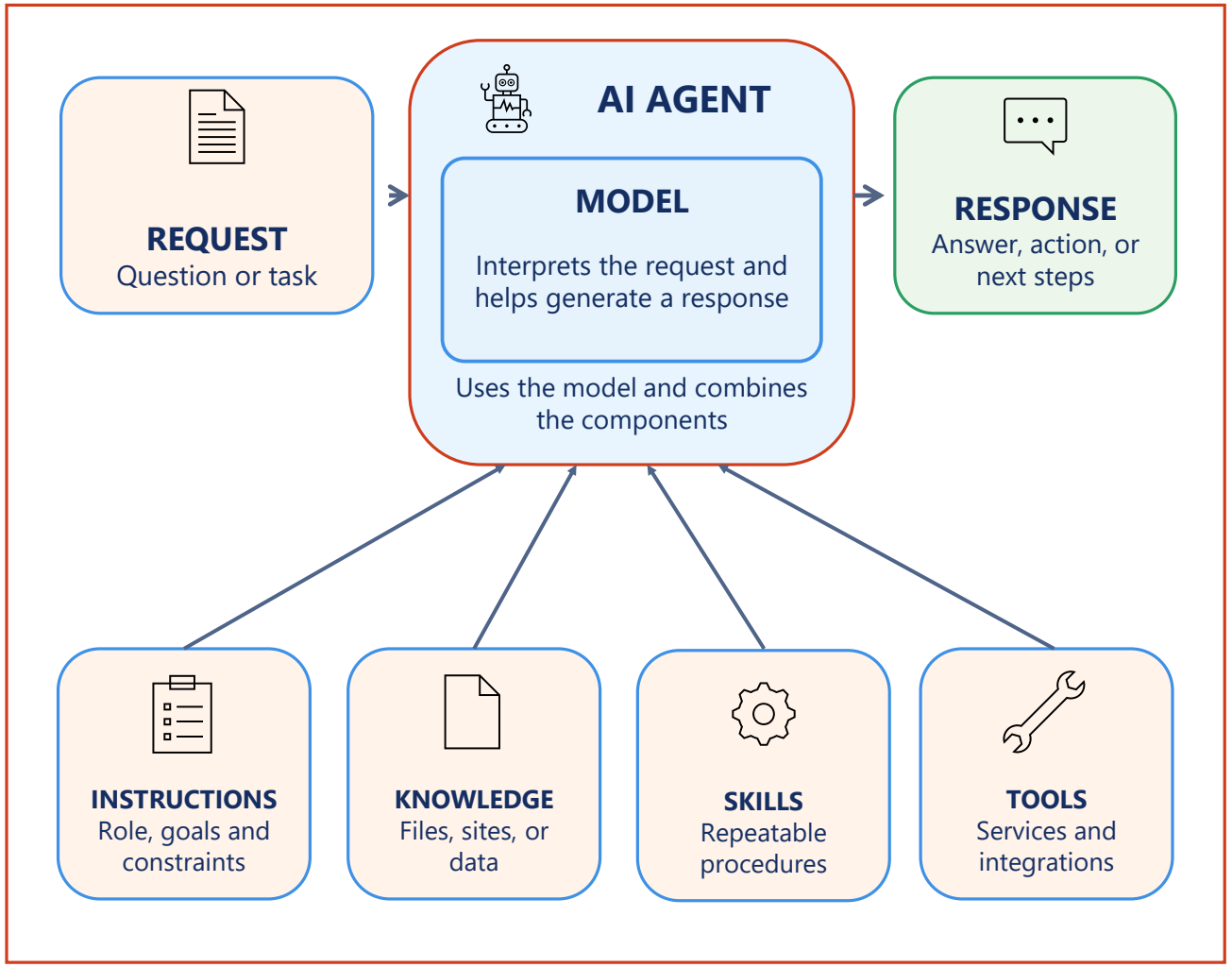


Twitter: https://twitter.com/robwindsor
LinkedIn: https://www.linkedin.com/in/rwindsor
Blog: https://robwindsor.hashnode.dev
YouTube: https://www.youtube.com/@RobWindsor
GitHub: https://github.com/rob-windsor

What Is an AI Agent?

An agent combines a model with instructions and may also use knowledge, tools, and skills.

- **Understands your request**
  The language model interprets the request, generates responses, and may suggest a next step.
- **Follows instructions**
  A role, goals and constraints shape how it responds.
- **Uses relevant knowledge**
  Files, sites, or other data can provide useful context.
- **Can use skills**
  Reusable procedures guide repeatable tasks.
- **Can use tools**
  Connected services can extend what it can do, subject to permissions and approval.

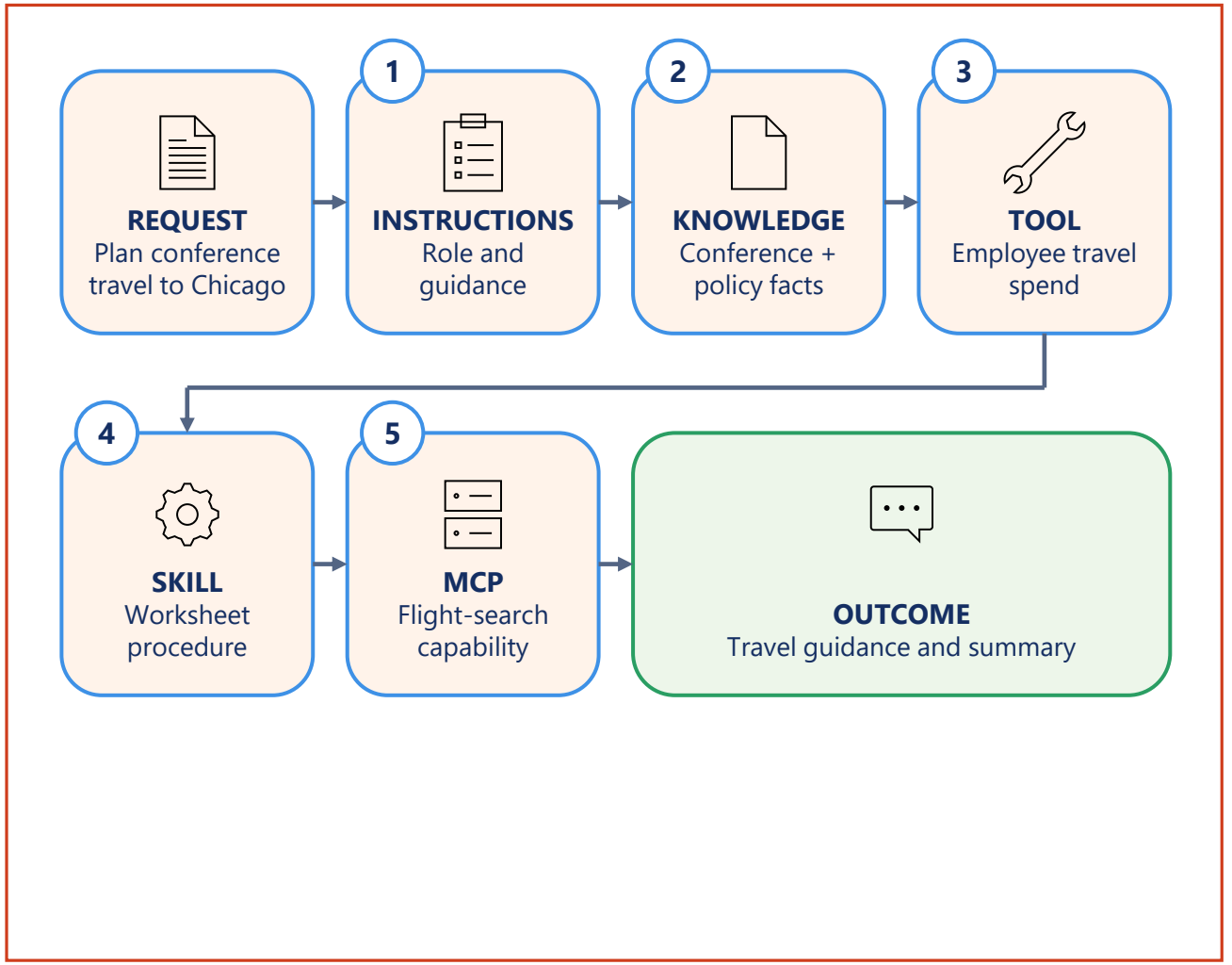


Building the Scenario

We will follow one conference travel request as the agent gains more capability.

- **Start with instructions**
  Give the agent a role and behavioral guidance.
- **Add knowledge**
  Retrieve conference details and company policy.
- **Add tools**
  Look up employee-specific travel spending.
- **Add a skill**
  Produce a structured travel approval worksheet.
- **Add MCP**
  Connect to a separate flight-search capability.

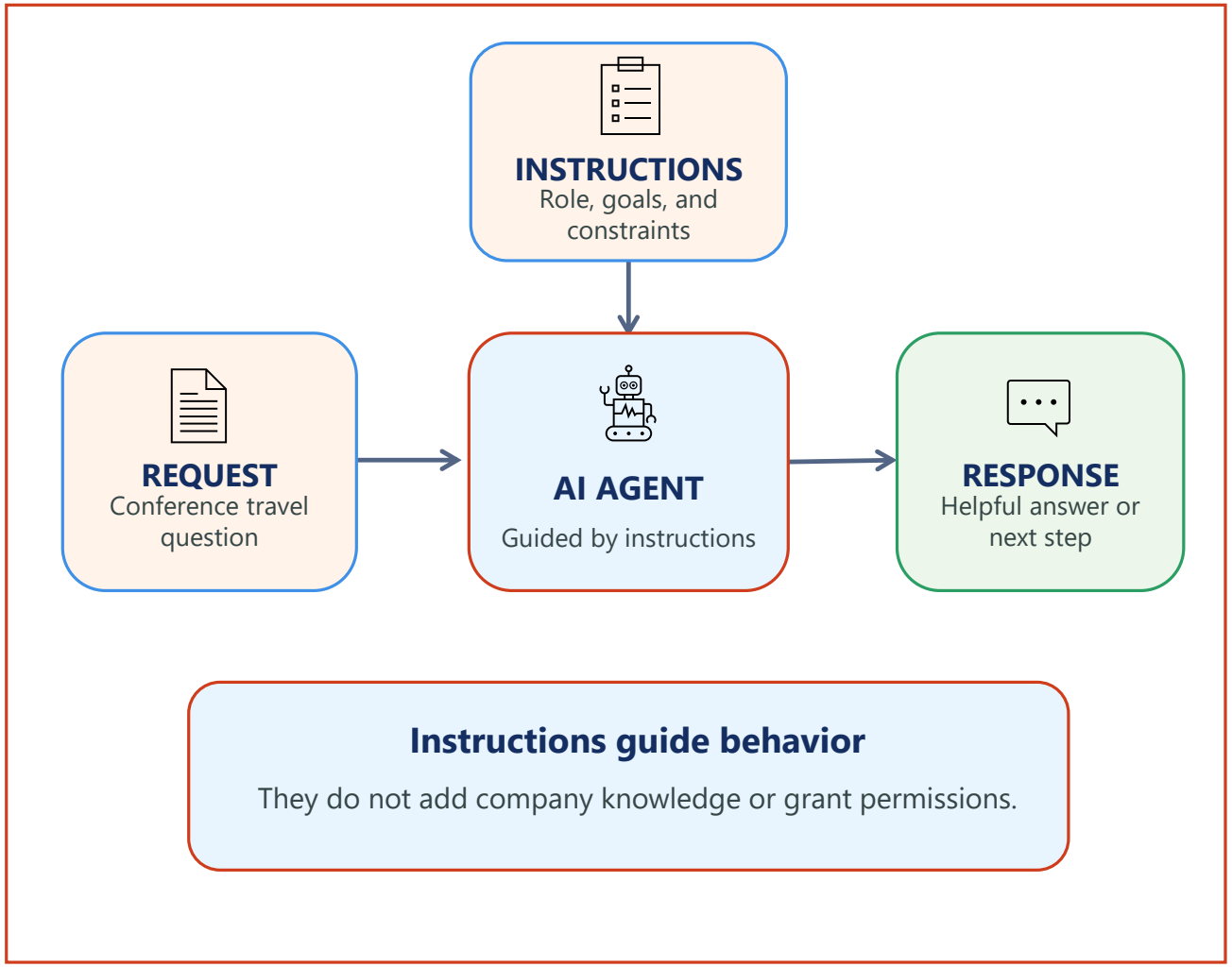
Same travel scenario, progressively richer outcomes.



Instructions Give an Agent a Job

Instructions define what kind of assistant an agent should be and how it should behave.

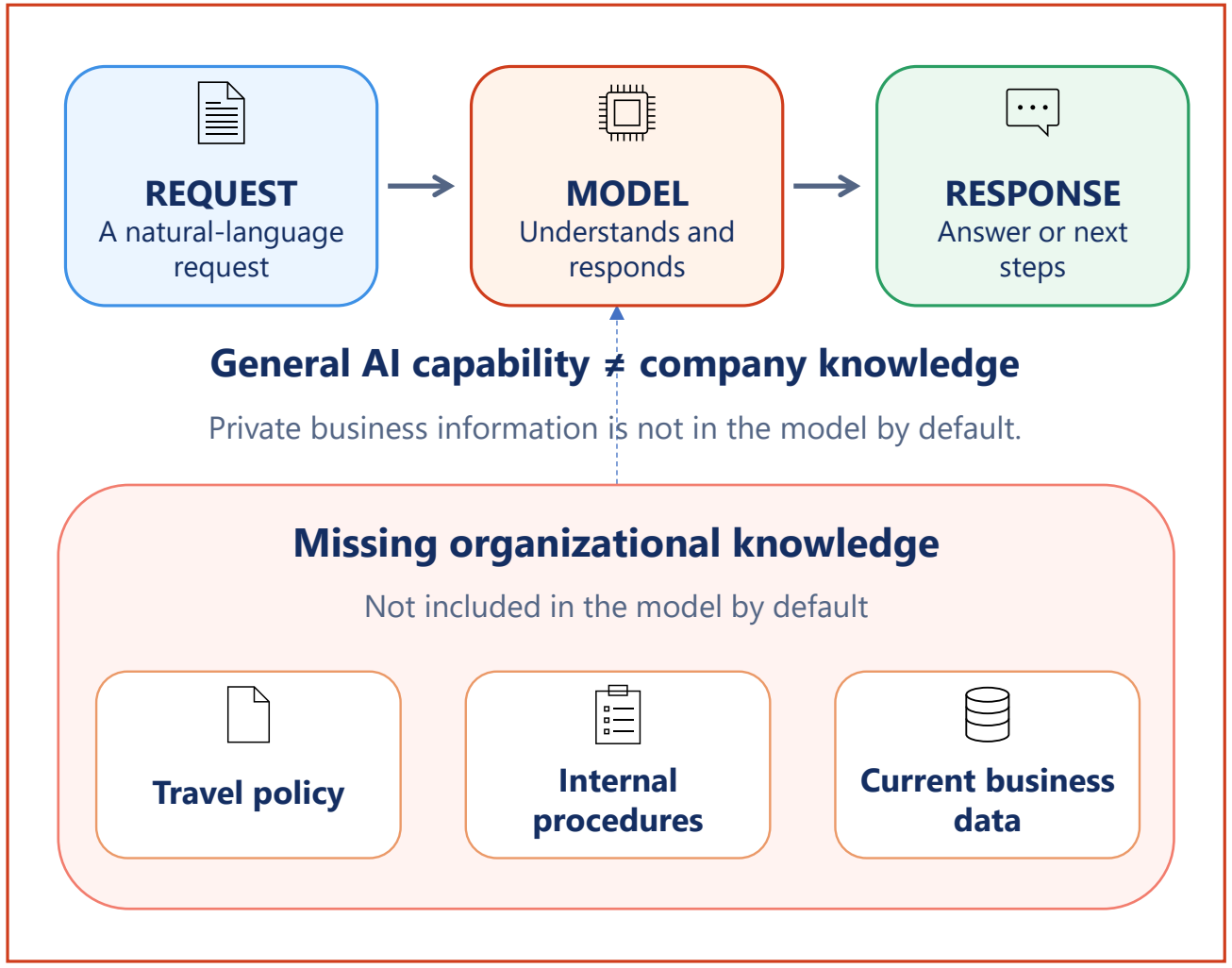
- **Defines the role**
  Sets the agent's purpose and role.
- **Sets expectations**
  Guides its response and priorities.
- **Reduces guessing**
  Prompts it to flag missing details, not guess.
- **Doesn't add knowledge or permissions**
  They do not supply private data or authorize actions.



The Model Is Not Your Company Knowledge Base

A language model can understand questions and generate answers. It doesn't automatically know your organization's private information.

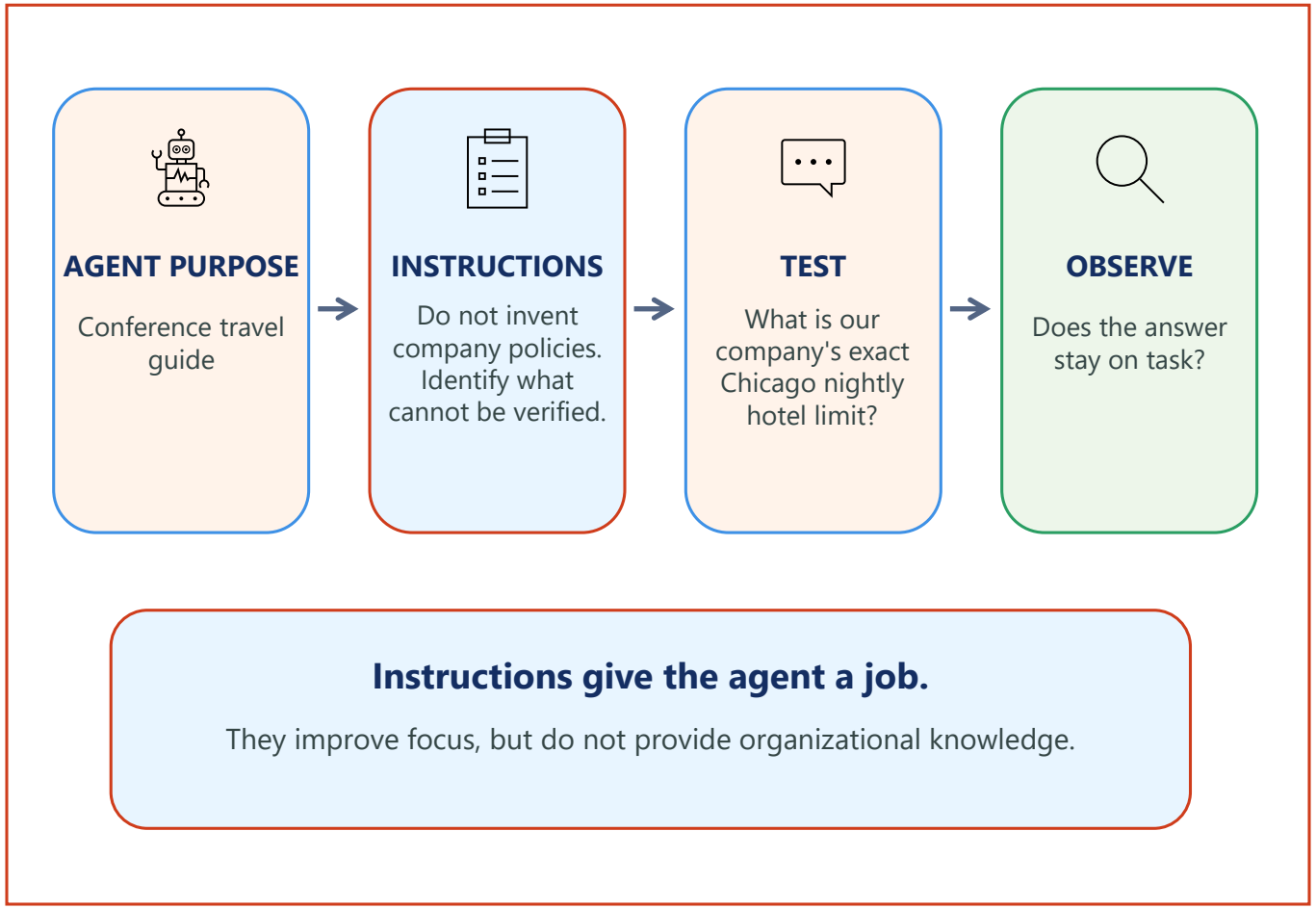
- **Understands the question**
  Interprets natural-language requests and generates a response.
- **Doesn't know private data**
  Company policies, procedures, and current business facts are not included by default.
- **Can still sound convincing**
  A fluent answer may look confident even when important information is missing.
- **Needs trusted sources**
  Provide relevant organizational knowledge and verify important claims.



Demo 1: Instructions

We'll use an agent built with C# and the Microsoft Agent Framework to see how instructions shape behavior.

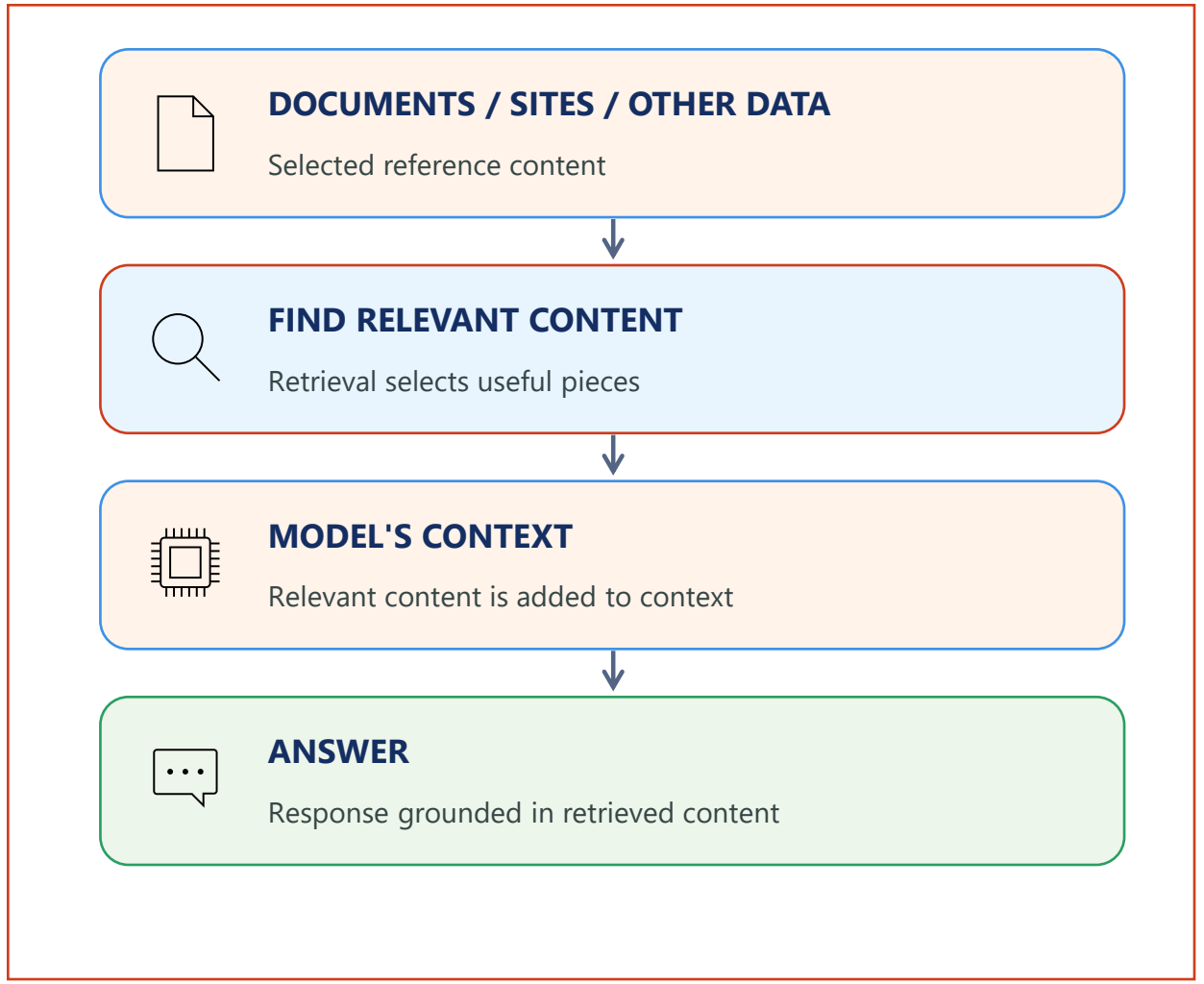
- **Agent purpose**
  Give the agent a clear job.
- **Focused instructions**
  Tell it how to answer and what not to guess.
- **Test question**
  Ask about conference travel policy.
- **Observe the result**
  Look for focused behavior.



Knowledge Is the Information the Agent Can Use

Knowledge is information available to an agent when needed.

- **Selected sources:** Files, SharePoint content, and other supported data.
- **Retrieval:** Finds pieces relevant to the current question.
- **Important limit:** Sources help ground answers, but do not guarantee source-only results.



What Does the Model Receive?

Context is assembled for each response and may include:

Current question: The latest request.

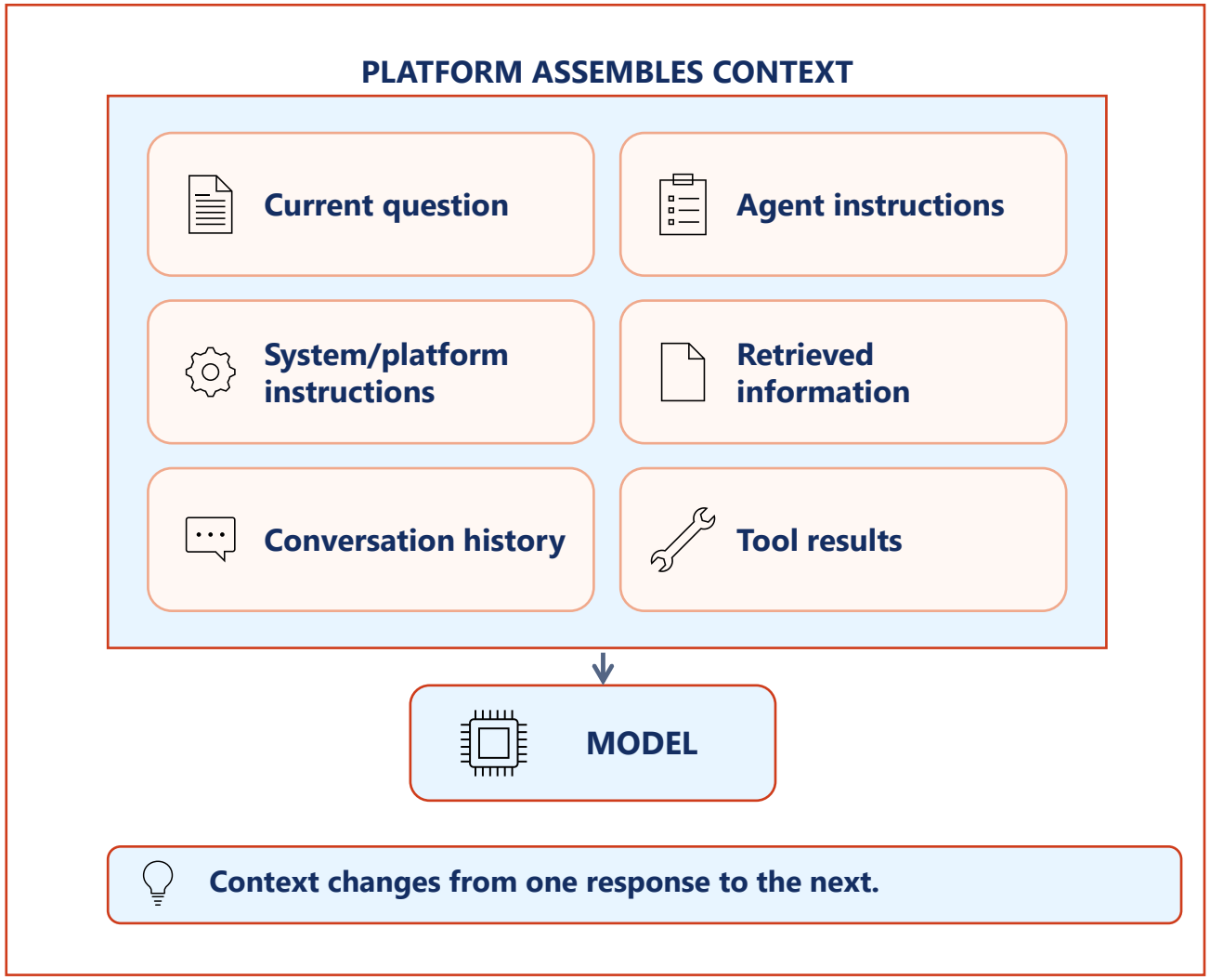
Agent instructions: Role, goals, and behavior guidance.

System/platform instructions: Additional directions from the product.

Retrieved information: Relevant content from files, sites, or other sources.

Conversation history: Relevant recent messages.

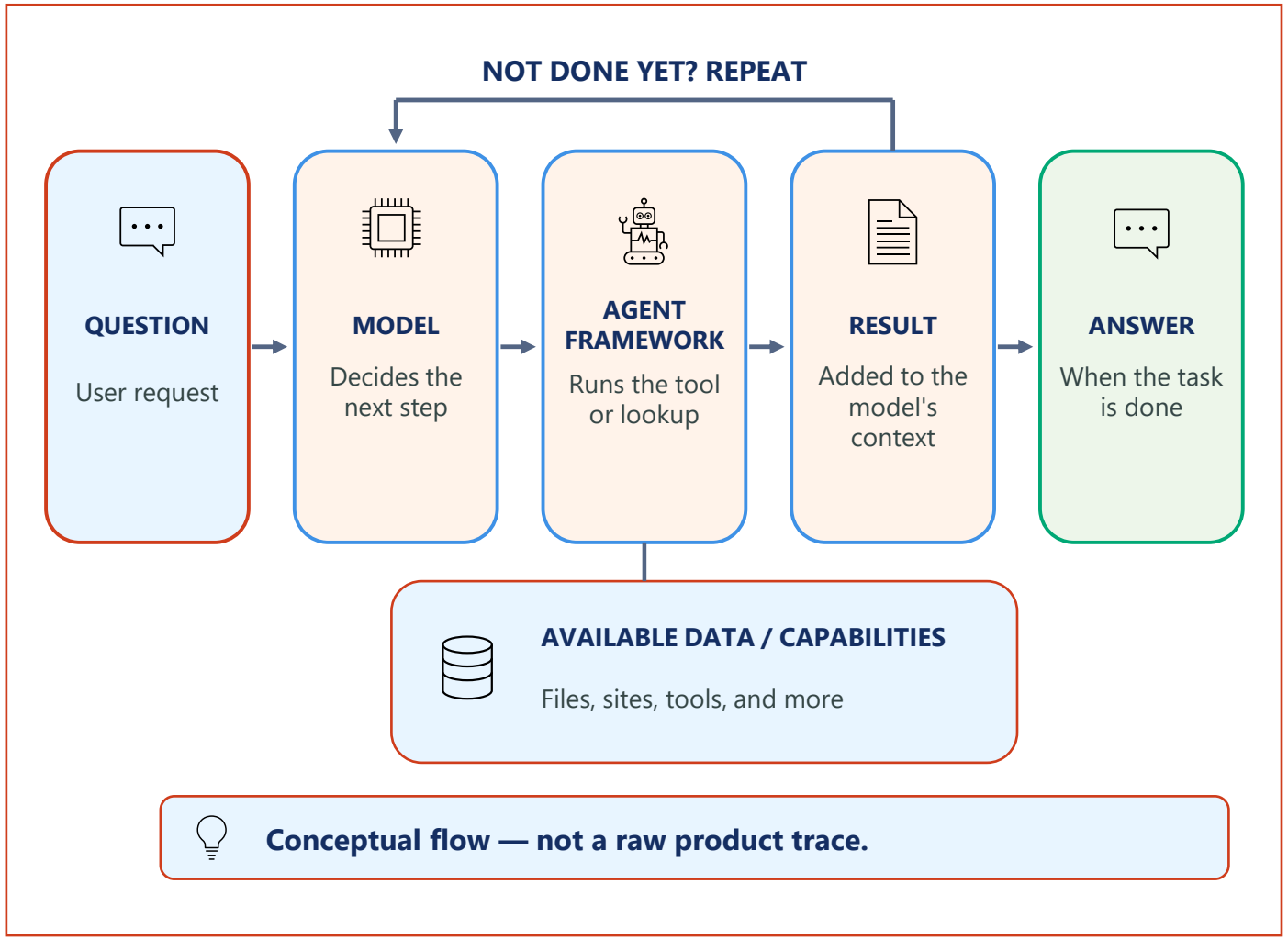
The platform selects and formats these inputs; you may not see all of them.



Orchestration Manages the Process

Orchestration is the behind-the-scenes coordination of an agent's activities. In a model-driven agent, this often works as a loop: take a step, check the result, and decide what happens next.

- The model decides the next step.
- The Agent Framework coordinates tool calls and returns their results to the model.
- The result is added to the model's context.
- Loop continues until the agent can respond or needs input from the user.



Demo 2: Give the Agent Company Knowledge

Add conference and policy documents, then verify the hotel selection.

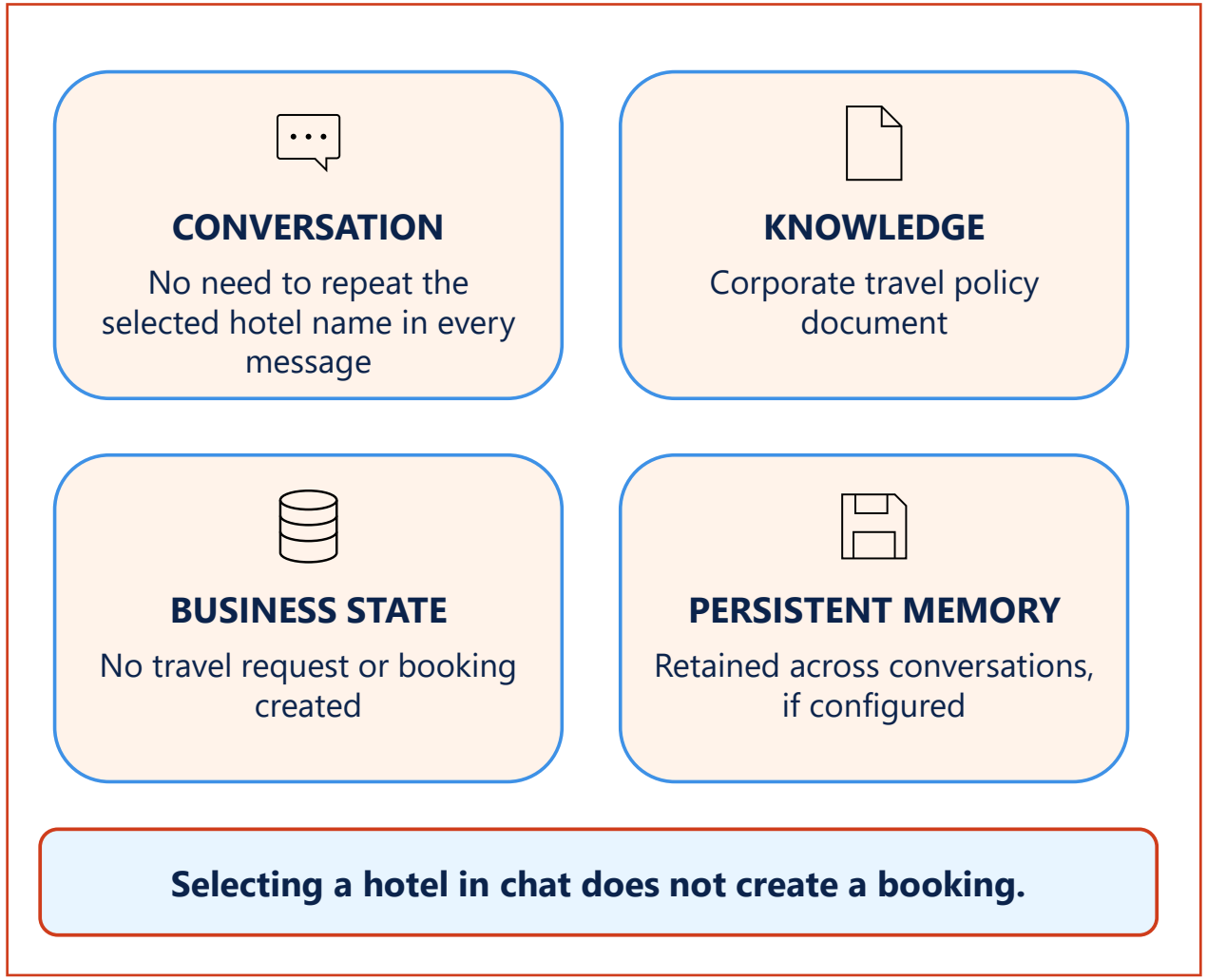
- **Retrieve company facts**
  Compare three hotel rates with the Chicago $275 nightly limit.
- **Confirm the chosen hotel**
  West Loop hotel is $225 per night, within the base-rate policy.
- **Use conversation context**
  The second prompt names the hotel but omits the nightly rate; the agent uses the $225 retrieved in turn 1 to calculate the cost..
- **Inspect the evidence**
  Retrieved passages and model context; 4 nights × $225 = $900 USD base lodging, before taxes and fees.



Conversation State Is Not Permanent Memory

Continuity in an agent involves several different kinds of information.

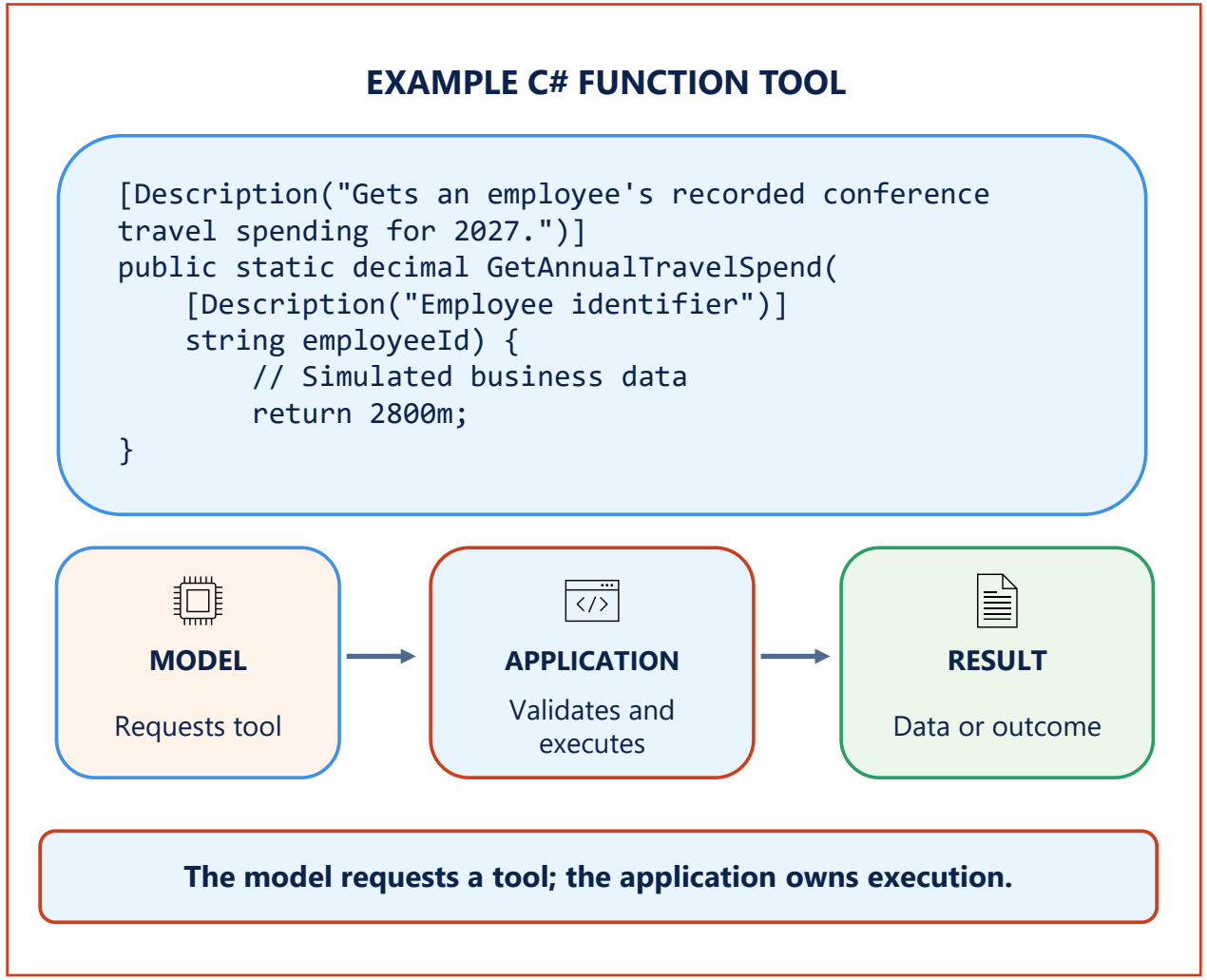
- **Conversation state**
  Retained for the current conversation; not automatically available in a new session.
- **Organizational knowledge**
  External source material that can be retrieved when relevant.
- **Business state**
  Information saved in a business system, such as a travel request or reservation.
- **Persistent memory**
  Selected information intentionally stored for use across conversations, when the application supports and configures it.



Tools Let an Agent Use External Capabilities

Tools expose operations the agent can request and the application can execute.

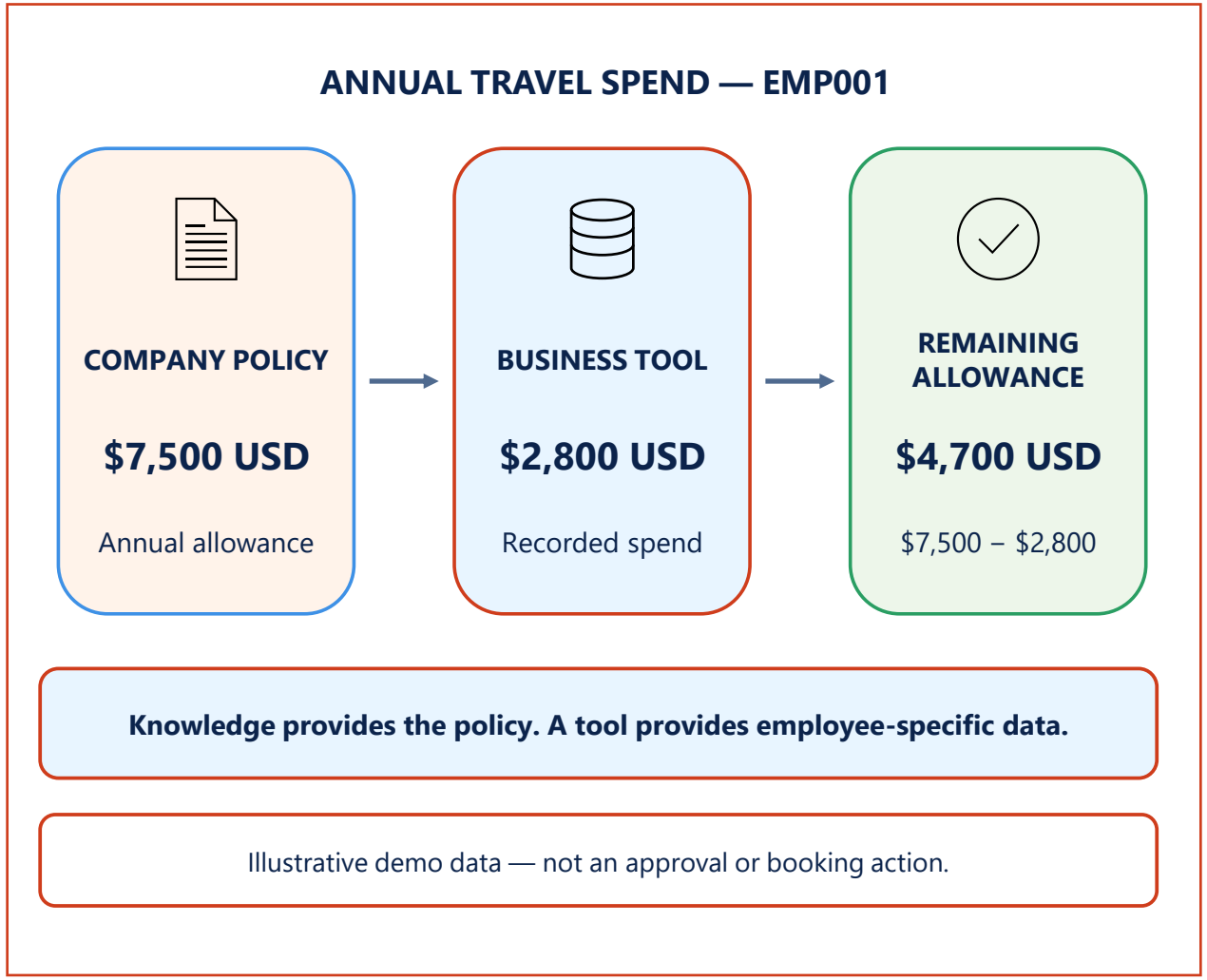
- **Tool contract**
  A name, description, inputs, and structured result.
- **Read operations**
  Retrieve information from a business system, such as an employee's recorded travel spending.
- **State-changing operations**
  Create or update information in a business system.
- **Application execution**
  Treat tool arguments as untrusted; validate before running.



Demo 3: Check Annual Travel Spend

Use company policy plus an employee-specific tool result to determine the remaining allowance.

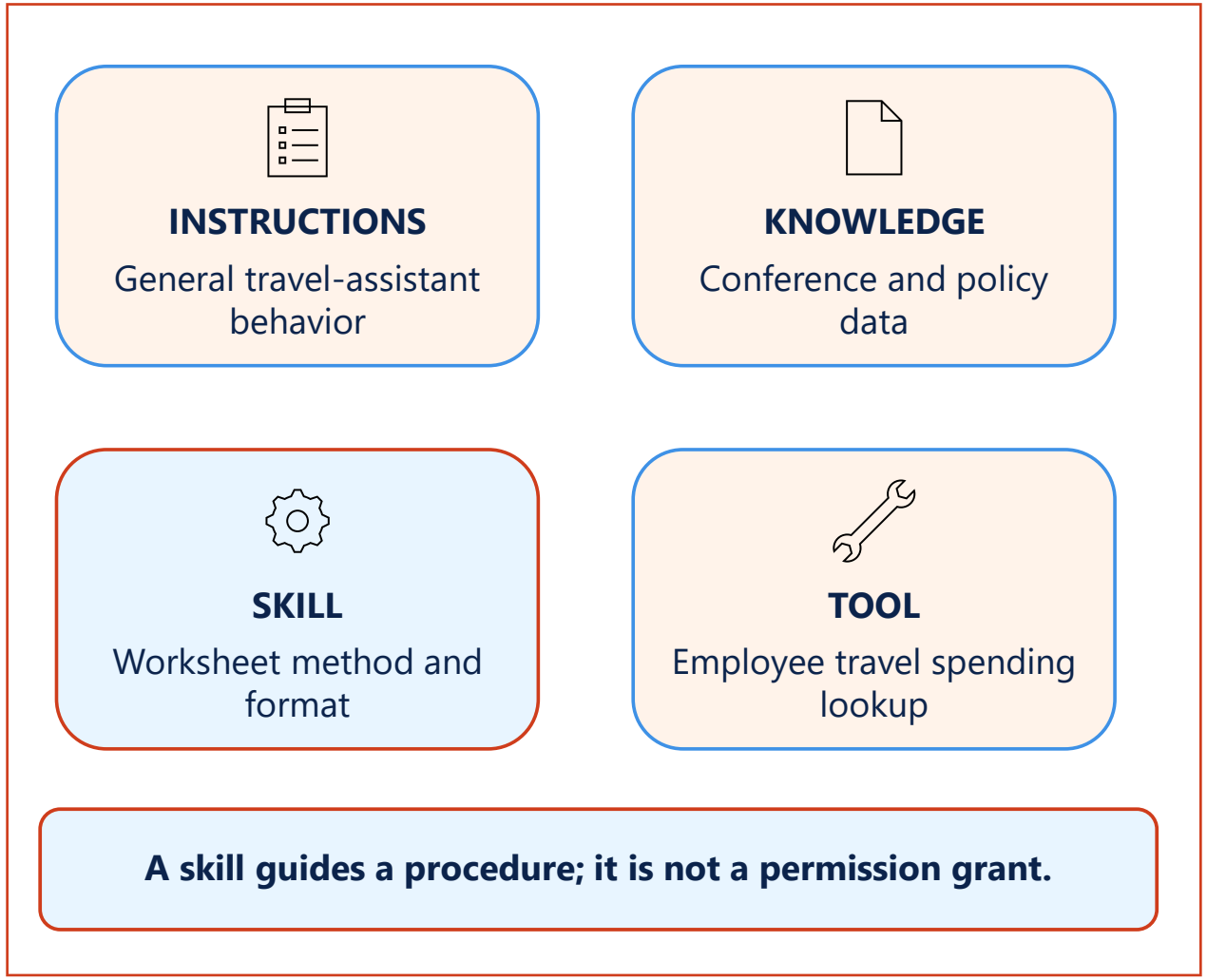
- **Retrieve company policy**
  The annual conference travel allowance is $7,500 USD.
- **Call the business tool**
  GetAnnualTravelSpend returns $2,800 USD for EMP001.
- **Calculate the remaining allowance**
  $7,500 − $2,800 = $4,700 USD remaining before this trip.
- **Inspect the evidence**
  Knowledge retrieval and the business tool both contribute to the final answer.



Skills Package Repeatable Tasks

A skill packages reusable task-specific guidance instead of repeating it in every prompt.

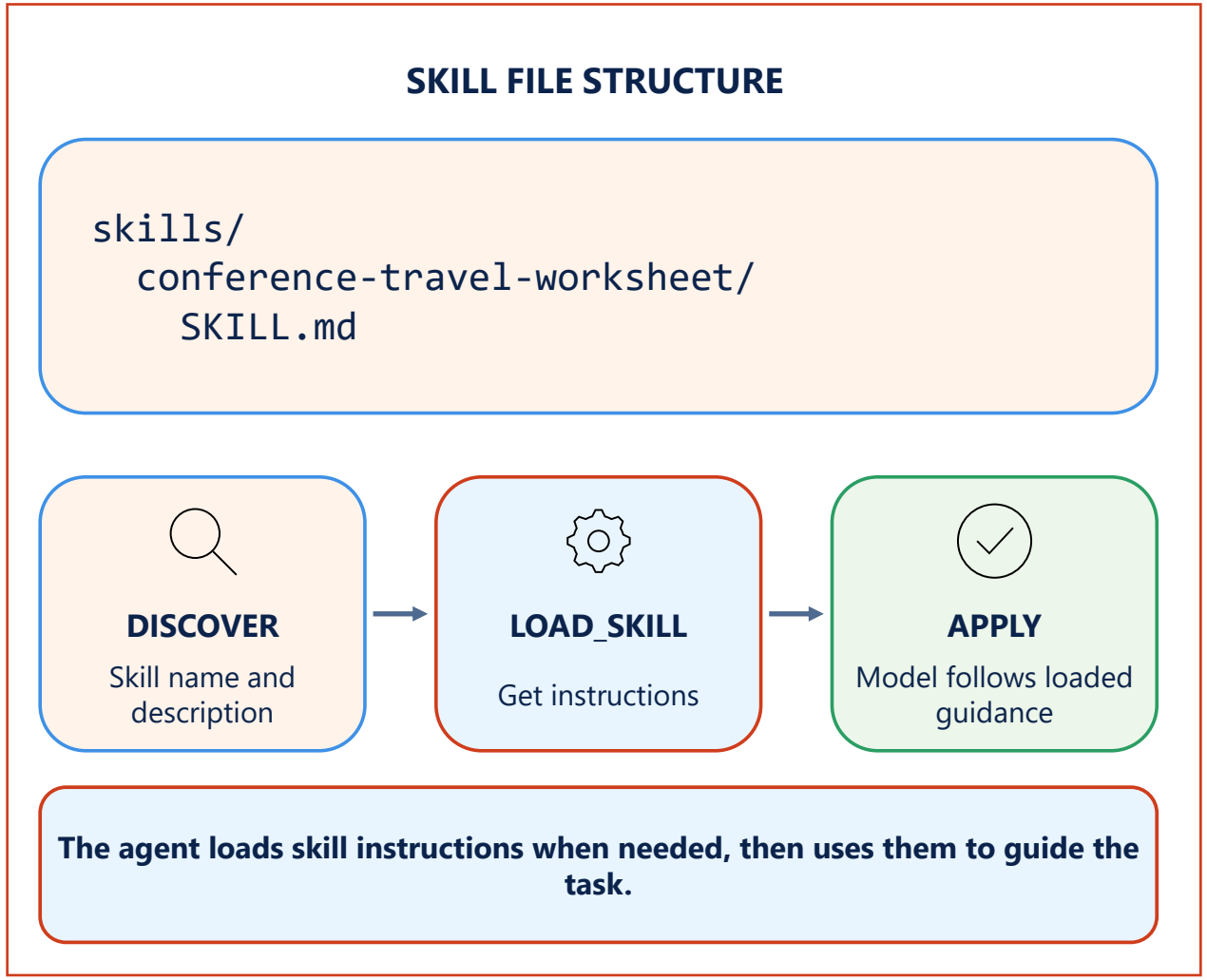
- **General instructions**
  Define the assistant's overall purpose and behavior.
- **Knowledge**
  Supplies facts from company documents and records.
- **Skill**
  Packages the procedure and output format for a worksheet.
- **Tool**
  Provides an executable capability, such as retrieving employee travel spending.



How Skills Are Loaded

The runtime exposes a skill description and can load detailed instructions when needed.

- **Skill package**
  SKILL.md defines the skill; optional files provide supporting resources.
- **Discovery**
  The agent receives skill descriptions without necessarily loading every skill's instructions.
- **Loading**
  When appropriate, the agent requests the skill's detailed instructions through load_skill.
- **Execution evidence**
  Inspect the actual tool call and result rather than inferring skill use from the response.



Demo 4: Create a Conference Travel Worksheet

Ask the agent to prepare a structured conference travel worksheet using company knowledge and employee spending data.

- **Request a worksheet**
  Ask the agent to prepare a conference travel worksheet.
- **Use the skill**
  Load Conference-Travel-Worksheet and follow its procedure.
- **Combine available information**
  Include the selected hotel, lodging cost, and remaining allowance.
- **Identify missing information**
  Airfare is not yet known, so the worksheet remains incomplete.



WORKSHEET REQUEST

User asks for a travel worksheet



SKILL APPLIED

Follow the worksheet procedure

WORKSHEET

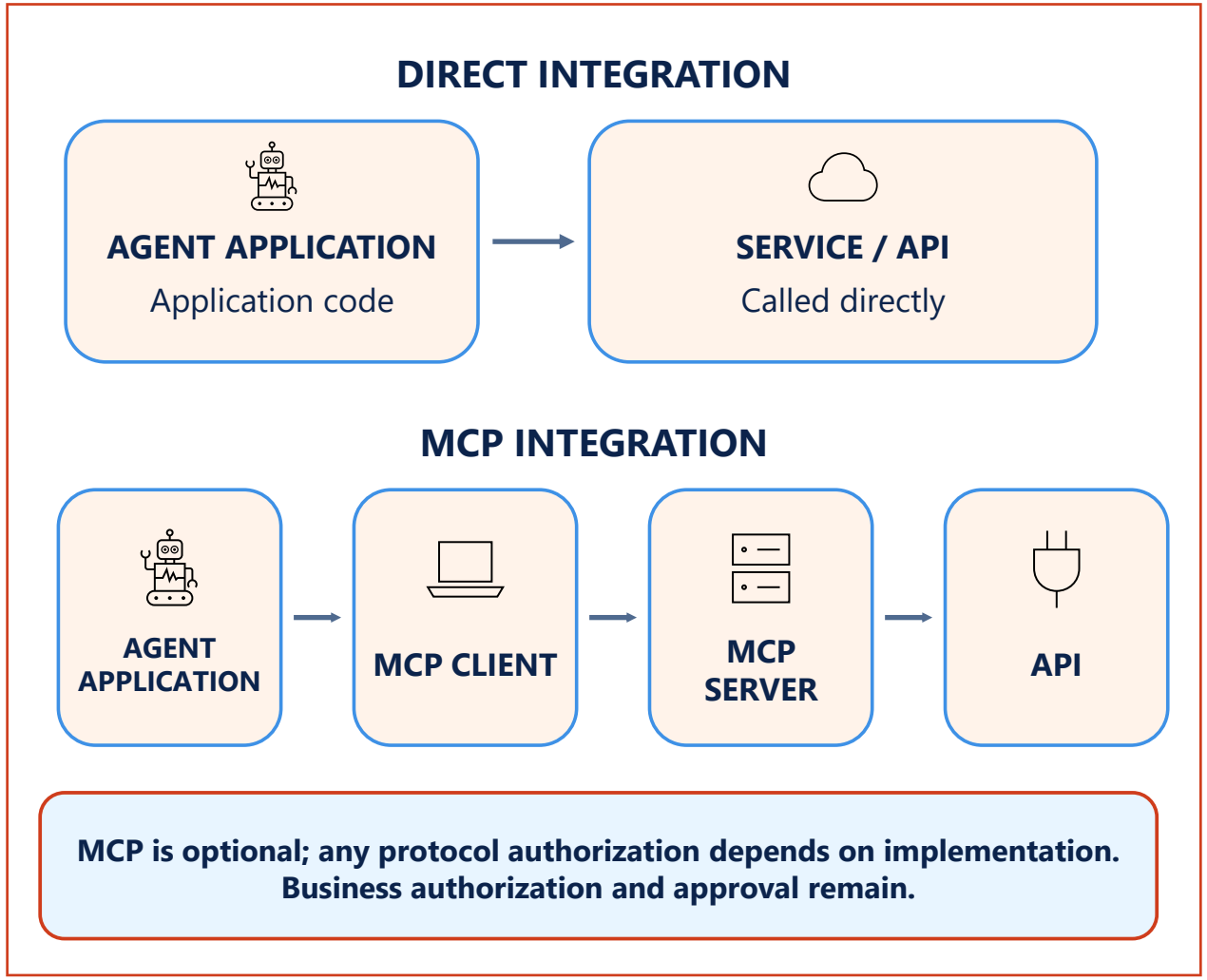
Hotel — West Loop, 4 nights × $225	$900
Airfare	Not yet determined
Annual allowance	$7,500
Prior spending	$2,800
Remaining before this trip	$4,700
Provisional balance	**$3,800**

The worksheet does not grant approval or create a travel-request record.
It organizes known information and highlights what's still needed for a travel request.

Model Context Protocol (MCP)

MCP is an optional standard integration boundary between an AI application and external capabilities.

- **Direct integration**
  Application calls a local function, API, or SDK directly.
- **MCP integration**
  An MCP client uses capabilities exposed by an MCP server.
- **What MCP standardizes**
  Discovery and invocation of tools, plus resources and prompts.
- **What remains outside**
  Application orchestration, downstream business rules, and approvals.



Demo 5: Search Flights through MCP

Use a flight-search tool exposed by a separate MCP server to find a flight matching the traveler's criteria.

- **Connect to MCP server**
  Discover tools exposed by a local Azure Functions host.
- **Search for flights**
  Toronto to Chicago, October 12–16, 2027.
- **Apply the traveler's criteria**
  Select the lowest-priced nonstop economy flight—not the cheapest fare overall.
- **Inspect the evidence**
  Confirm SearchFlights execution in the Functions host logs.



Demo 5: Update the Worksheet

Use the selected MCP flight result to update the worksheet and recalculate the known trip cost.

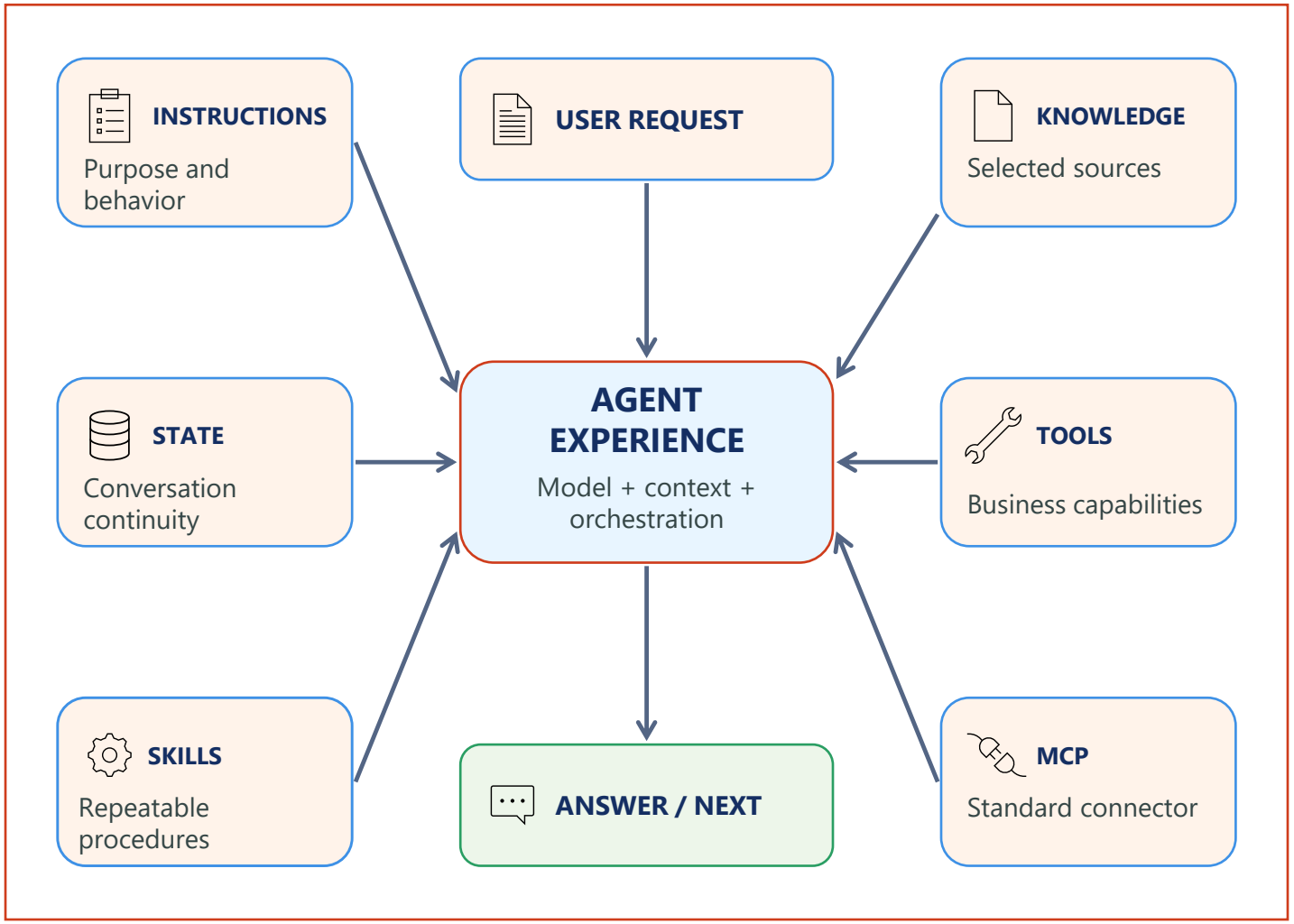
- **Add the selected flight**
  Insert CF-101 as the chosen lowest-priced nonstop economy option: $690 USD base fare.
- **Recalculate known trip costs**
  Hotel ($900) + airfare ($690) = $1,590 before taxes and fees.
- **Update the remaining allowance**
  $4,700 remaining before this trip becomes $3,110 after current known costs.
- **Keep the limits in context**
  Registration, ground transportation, taxes, fees, and final availability are still unknown.



Put the Anatomy Back Together

The same components support a conference-travel request, even when the platform hides the implementation.

- Platform orchestration manages retrieval, model interaction, and progression.
- Instructions, knowledge, state, and skills shape the experience.
- Tools may reach approved systems directly or through MCP, when supported.



Resources

Microsoft Agent Framework — Documentation
https://learn.microsoft.com/en-us/agent-framework

Build Your First Agent with C#
https://learn.microsoft.com/en-us/agent-framework/get-started/your-first-agent

Microsoft Agent Framework — GitHub Repository
https://github.com/microsoft/agent-framework

Microsoft Agent Framework — .NET Samples
https://github.com/microsoft/agent-framework/blob/main/dotnet/samples/README.md

Agent Skills in Microsoft Agent Framework
https://learn.microsoft.com/en-us/agent-framework/agents/skills

Model Context Protocol (MCP)
https://modelcontextprotocol.io